

Laikipia County Job Recruitment System

Full Project Documentation

Generated from the current frontend and backend codebase.

1. Project Overview

The Laikipia County Job Recruitment System is a web-based recruitment management platform built with a Spring Boot backend and a React/Vite frontend. It supports the full hiring lifecycle from department staffing requests through vacancy publication, applicant onboarding, application processing, shortlisting, interviews, final selection, notifications, and email communication.

- Active backend: cgl (Spring Boot, JPA, Security, Mail, JWT)
- Active frontend: frontend (React, Vite, Router, Axios, jsPDF)
- Persistence: MySQL database
- Supporting storage: local uploads directory for applicant documents
- Communication: in-app notifications and SMTP email delivery

2. Objectives

1. Digitize and centralize county recruitment workflows.
2. Provide controlled role-based access for internal staff.
3. Allow applicants to discover vacancies, apply online, and track progress.
4. Maintain official records and printable reports for recruitment processes.
5. Improve applicant communication using notifications and email.
6. Secure system access through JWT-based authentication and email verification.

3. Technology Stack

Layer	Technologies
Backend	Java 21, Spring Boot 4.1, Spring Web MVC, Spring Security, Spring Data JPA, Lombok, Spring Mail, JWT, OpenAPI/Swagger
Frontend	React 19, Vite 8, React Router 7, Axios, Tailwind CSS, Lucide React, jsPDF, jsPDF-AutoTable
Database	MySQL
Storage	Local file storage under uploads/documents
Authentication	JWT bearer tokens with role-based authorization

4. Active Project Structure

- cgl/: active Spring Boot backend with controllers, services, models, repositories, security, and email verification logic.
- frontend/: active React frontend with public, applicant, and admin/staff pages.
- uploads/: storage for uploaded application documents.
- laikipiaJobs/: appears to be an extra frontend scaffold and is not the active integrated frontend in current use.

5. Architecture Overview

The frontend issues REST requests to the backend through Axios. The backend validates authentication and role access, executes business logic in service classes, persists data via JPA repositories, and returns JSON responses. Uploaded files are stored on disk, while relational business data is stored in MySQL.

1. User interacts with frontend page or form.

2. Frontend sends an HTTP request to a backend endpoint.
3. Security layer validates token and role where required.
4. Controller delegates work to a service.
5. Service performs validation, workflow logic, persistence, and side effects such as notifications or email.
6. Backend returns response payload for rendering or redirect decisions.

6. User Roles and Responsibilities

Role	Main Responsibilities	Cannot Do
SUPER_ADMIN	Create staff users, manage departments, access most admin modules and reports, oversee system configuration areas through the UI.	Core hiring decisions such as approving recruitment requests, shortlisting, interview scheduling, or final candidate selection.
CPSB_ADMIN	Approve or reject recruitment requests, create vacancies, oversee recruitment workflow, participate in shortlist/final selection functions.	General system administration reserved for superadmin.
DEPT_HEAD	Submit departmental recruitment requests and monitor request records.	Approve requests or perform downstream hiring actions.
HR_OFFICER	Review applications, update application states, shortlist candidates, schedule interviews, assign panel members, complete interviews.	Approve recruitment requests or finalize final appointment decisions.
PANEL_MEMBER	View assigned interviews and submit interview scores.	Vacancy management, shortlisting, or final hiring decisions.
APPLICANT	Register, verify email, build profile, apply for jobs, upload documents, view notifications and statuses.	Any staff or admin operation.

7. Authentication and Verification Flow

Authentication is handled through JWTs, while email verification uses one-time 6-digit codes sent through SMTP. Verification is required for new applicants and newly created non-superadmin staff users before first login.

1. Applicant registration creates a Users record with role APPLICANT and emailVerified = false.
2. Superadmin-created staff accounts are also created with emailVerified = false.
3. A verification code is generated, stored in EmailVerificationToken, and sent by email.
4. The user enters email and code on /verify-email.
5. On successful verification, emailVerified becomes true and the code is marked used.
6. User then logs in and receives a JWT token.

Current continuity rule: the codebase presently allows SUPER_ADMIN login even if emailVerified is false, so legacy superadmin access is not lost.

8. Security Model

- Spring Security is configured as stateless using JWTs.
- Method-level security is enabled with @PreAuthorize.
- Public endpoints include /auth/**, GET /jobs/**, GET /departments/allDepartments, and Swagger docs.
- All other endpoints require authentication and role authorization.
- Frontend route guards separate public, applicant, and admin areas.

9. Core Functional Modules

Module	Purpose	Primary Actors
Authentication	Registration, login, JWT issuance, email verification, resend verification code.	All users

Module	Purpose	Primary Actors
Applicant Profile	Capture personal and qualification details required for applications.	Applicant
Vacancy Management	Create, open, close, and delete vacancies.	CPSB Admin, Super Admin (view/manage access)
Applications	Submit, list, and update job applications.	Applicant, HR Officer, CPSB Admin, Super Admin (view)
Documents	Upload and review supporting files attached to applications.	Applicant, HR Officer, CPSB Admin, Super Admin (view)
Recruitment Requests	Initiate and approve staffing demand.	Dept Head, CPSB Admin, Super Admin (view)
Shortlisting	Move candidates into shortlist stage and produce shortlist records.	HR Officer, CPSB Admin
Interviews	Schedule interviews, assign panels, track interview status.	HR Officer, Panel Member, CPSB Admin, Super Admin (view)
Interview Scoring	Submit and review interview score records.	Panel Member and relevant staff
Final Selection	Record selected candidates and appointment progression.	CPSB Admin, HR Officer (view), Super Admin (view)
Notifications	In-app communication of workflow events.	All users
Email Messaging	Verification and applicant outcome messages.	Applicants and staff users

10. End-to-End Recruitment Workflow

1. Department Head submits a recruitment request.
2. CPSB Admin reviews and approves or rejects the request.
3. CPSB Admin creates a vacancy from approved staffing demand.
4. Applicant registers, verifies email, and logs in.
5. Applicant completes profile and applies for an open vacancy.
6. Applicant uploads supporting documents if required.
7. HR Officer or CPSB Admin reviews applications.
8. Rejected applications trigger unsuccessful notification and email.
9. Eligible candidates are shortlisted and receive shortlist communication.
10. HR Officer schedules interviews and assigns panel members.
11. Panel members score candidates.
12. CPSB Admin records final selection and appointment progression.
13. Selected applicants receive acceptance notification and email.

11. Applicant-Side Features

- Public vacancy browsing on / and /vacancies.
- Account creation on /register and verification on /verify-email.
- Applicant dashboard at /dashboard.
- Profile management via /profile.
- Document upload and management via /documents.
- Application history via /applications.
- Personal notifications via /notifications.

12. Staff/Admin-Side Features

- Admin dashboard at /admin with role-filtered navigation.

- User management for superadmin via /admin/users.
- Department management via /admin/departments.
- Recruitment request handling via /admin/recruitment.
- Vacancy management via /admin/vacancies.
- Application reporting via /admin/applications.
- Shortlist reporting and operations via /admin/shortlists.
- Interview scheduling and reporting via /admin/interviews.
- Final selection reporting and operations via /admin/selections.

13. Data Model Overview

Entity	Purpose	Key Relationships
Users	Authentication identity, role, email verification state.	Referenced by applicant profiles, creators, approvers, notifications, and verification tokens.
Applicant	Extended applicant profile including national ID, education, county, and experience.	One-to-one with Users.
Departments	Organizational department master record.	Linked to recruitment requests and vacancies.
RecruitmentRequest	Formal staffing demand request.	Belongs to a department and requesting user.
JobVacancy	Published job opening.	Linked to department, creator, and optionally a recruitment request.
Applications	Applicant submission for a vacancy.	Belongs to one applicant and one vacancy.
ApplicationDocument	Uploaded file attached to an application.	Belongs to an application.
Shortlist	Shortlisting decision record.	One application per shortlist record.
Interview	Scheduled interview.	One-to-one with application.
InterviewPanel	Assignment of panel members to interviews.	Links interview and staff users.
InterviewScore	Panel evaluation record.	Linked to interview and scoring member.
FinalSelection	Final candidate selection record.	One-to-one with application.
Notification	In-app user notification.	Belongs to one user.
EmailVerificationToken	One-time email verification code, expiry, and usage record.	One-to-one with user.

14. Important Domain States

- Roles: SUPER_ADMIN, CPSB_ADMIN, DEPT_HEAD, HR_OFFICER, PANEL_MEMBER, APPLICANT.
- Recruitment request status: PENDING, APPROVED, REJECTED.
- Vacancy status: OPEN, CLOSED, FILLED.
- Application state: SUBMITTED, UNDER_REVIEW, REJECTED, SHORTLISTED, INTERVIEW, SELECTED.
- Interview status: SCHEDULED, COMPLETED, CANCELLED.
- Appointment status: PENDING_APPOINTMENT, APPOINTED, DECLINED.
- Supported document types: CV, DEGREE, KCSE, ID_COPY, CERTIFICATE, DRIVING_LICENSE.

15. Backend API Summary

Area	Representative Endpoints
Authentication	POST /auth/register, POST /auth/login, POST /auth/verify-email, POST /auth/resend-verification-code
Applicant	POST/PUT/GET /applicant/profile
Jobs	GET /jobs/allVacancies, GET /jobs/{id}, POST /jobs, PUT /jobs/{id}/open, PUT /jobs/{id}/close, DELETE /jobs/{id}
Departments	GET /departments/allDepartments, POST /departments/create, PATCH /departments/{id}, DELETE /departments/{id}
Applications	POST /applications/apply, GET /applications/my, GET /applications/vacancy/{vacancyId}, GET /applications/all, PUT /applications/{id}/status
Documents	POST /documents/upload, GET /documents/application/{applicationId}, GET /documents/my/{applicationId}
Notifications	GET /notifications/my, GET /notifications/unread, PUT /notifications/{id}/read
Recruitment	POST /recruitment/submit, PUT /recruitment/{id}/approve, PUT /recruitment/{id}/reject, GET /recruitment/pending, GET /recruitment/all, GET /recruitment/department/{departmentId}
Shortlist	POST /shortlist, GET /shortlist/vacancy/{vacancyId}
Interviews	POST /interviews/schedule, POST /interviews/panel, GET /interviews/my, GET /interviews/status/{status}, PUT /interviews/{id}/complete
Interview Scores	POST /scores, GET /scores/interview/{id}, GET /scores/interview/{id}/average
Selections	POST /selections, PUT /selections/{id}/appoint, GET /selections/vacancy/{vacancyId}
Admin	POST /admin/users/create

16. Notifications and Email Behavior

- In-app notifications are used for application receipt, recruitment request decisions, shortlist events, interview invitations, and final selection outcomes.
- Email is used for verification codes, shortlist communication, acceptance communication, and unsuccessful application communication.
- Verification codes are single-use and time-bound.

17. Configuration Overview

- Datasource configured for MySQL with Hibernate schema update enabled.
- JWT secret and expiration configured in application.properties.
- Upload directory configured as ./uploads/documents.
- SMTP mail settings configured for email sending.
- Frontend base URL configured for auth-related flows.

Security recommendation: secrets and passwords currently appear in source configuration and should be externalized to environment variables or a secure secrets store.

18. Operational Recommendations

1. Externalize secrets from application.properties.
2. Introduce formal DB migrations such as Flyway or Liquibase.
3. Add support tooling for resending verification codes from admin user management.
4. Consider persistent auth strategy if login state should survive page refresh.
5. Reduce frontend bundle size using route-level code splitting.

6. Add automated tests around verification, login, and recruitment state transitions.

19. Conclusion

The project implements a complete county recruitment workflow with clear separation between applicant operations, internal recruitment operations, and system administration. The current architecture is suitable for structured public-sector recruitment processes and can be strengthened further through improved secret management, formal migrations, and broader automated testing.